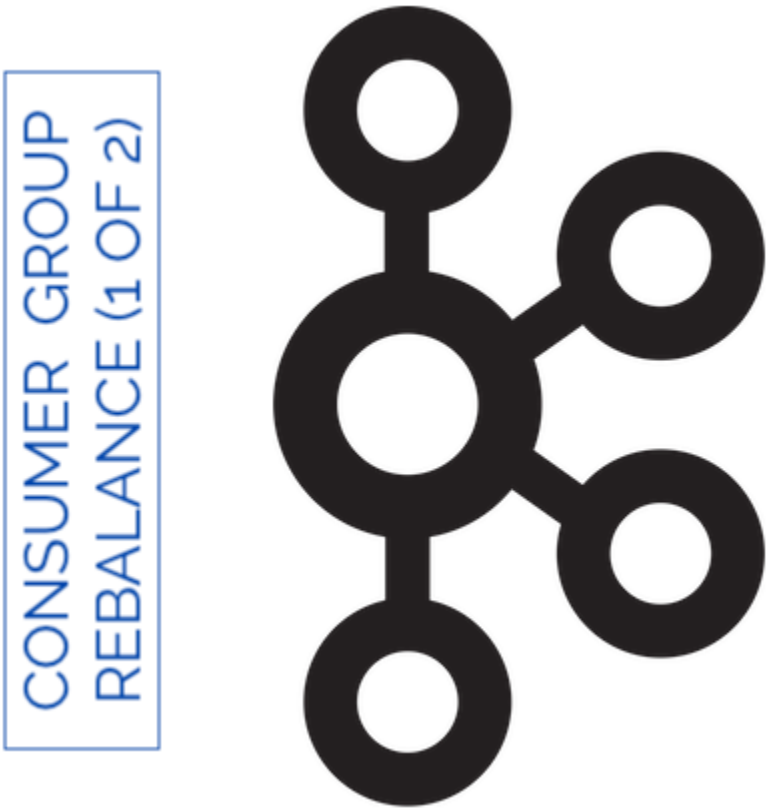
Kafka Consumer Group Rebalance - Part 1 of 2

Rob Golder - April 2022



Introduction

Consumer groups are an important characteristic of Kafka's distributed message processing for managing consumers and facilitating the ability to scale up applications. They group together consumers for any one topic, and the partitions within the topic are assigned across these consumers. Consumer group rebalance can be triggered by a number of factors as the participants of the group change, which leads to the reassignment of partitions across the consumers. During a rebalance message processing is paused, impacting throughput.

This is the first of a two part article that details the behaviour of consumer group rebalance with the Apache Java client. While many of the concepts are the same across different client libraries, there are also differences in terms of configuration options, rebalance behaviour, and supported rebalance strategies and features.

In this first part of the article the role of consumer groups, the consumer group rebalance, and the triggers that cause rebalances are covered. The configurations that impact both the duration of the rebalance and when a rebalance is triggered are detailed. In the second part the impact on the application's message processing during the rebalance is covered along with the rebalance strategies that can be applied. The options to reduce unnecessary rebalances and to mitigate the impacts of rebalances are explored.

Consumer Groups

When an application has a Kafka consumer implemented to consume messages from a topic, that consumer belongs to a consumer group. Within the consumer group, consumers are assigned topic partitions from which to consume. Group membership is managed on the broker side, and partition assignment is managed on the client side. The broker has no knowledge of what the resources are and how they are assigned amongst the consumers. This is a good example of why the Kafka client is considered a thick client. For more on this read about the role of the Kafka client.

The consumer is configured with a group.id, so that any other consumer instances with the same group.id will belong to the same consumer group. This facilitates the ability to scale up consumers, and this coupled with increasing the number of partitions in a topic provides a mechanism to increase message throughput.

The Group Coordinator manages the consumer group and the consumers. This is a Kafka component that lives on the broker side. It will make one consumer the lead, and this will be responsible for computing the topic partition assignments. These are returned to the Group Coordinator which then assigns the partitions to the consumers.

Given a single application instance, with a consumer with a group.id of 'foo' listening to a particular topic, and that topic has six partitions, then the consumer will poll for messages across all six partitions.

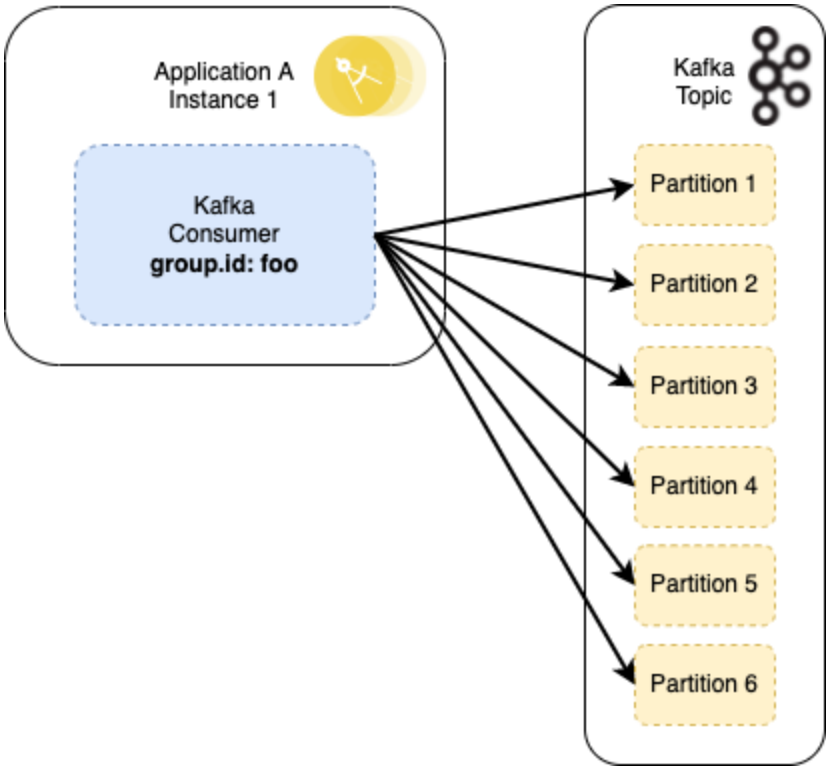


Figure 1: Single consumer group with one consumer

Now a second instance of the application is started. This therefore starts a second consumer instance with the same group.id of ‘foo’. The second consumer instance sends a JoinGroup request to the Group Coordinator, and the partitions are reassigned across the consumer group to spread the load. With two members in the consumer group, three partitions are assigned to each consumer instance.

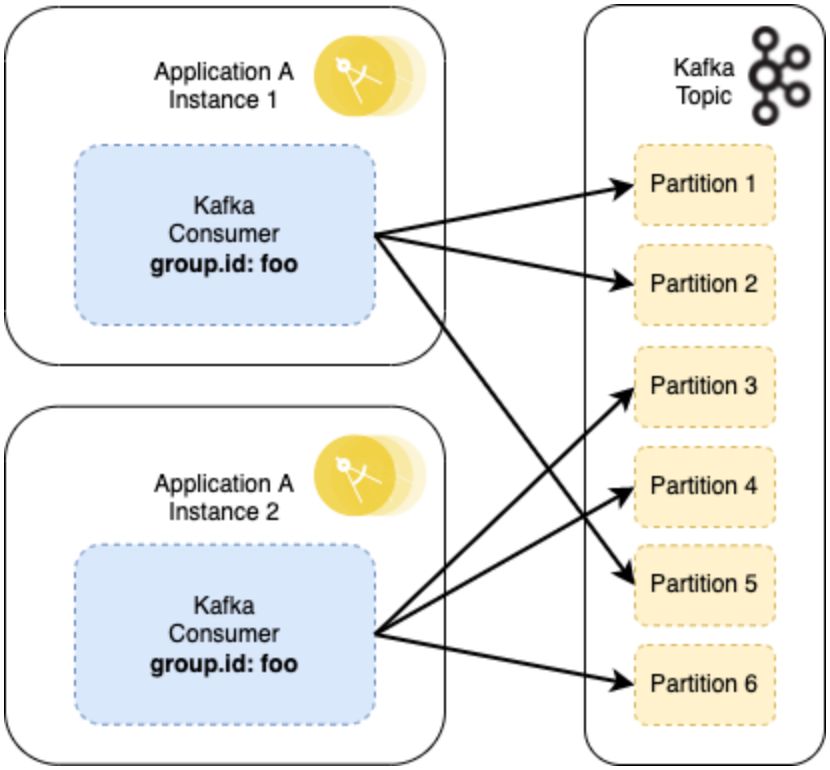


Figure 2: Single consumer group 'foo' with two consumers

Start a third application, and the partitions are again reassigned by the Group Coordinator, with each consumer now polling for messages from two partitions each.

If there are more consumer instances than partitions then those extra consumers will have no partitions assigned. A topic partition will only ever have one consumer listening to it from a given consumer group. So a consumer group composed of five consumers listening to a topic with three partitions will have two idle consumers.

If a consumer is started with a different group.id configured (as would be the case for a different service), and it is listening to the same topic, then this will be part of a separate consumer group. Its partition assignments are independent of those from any other consumer group.

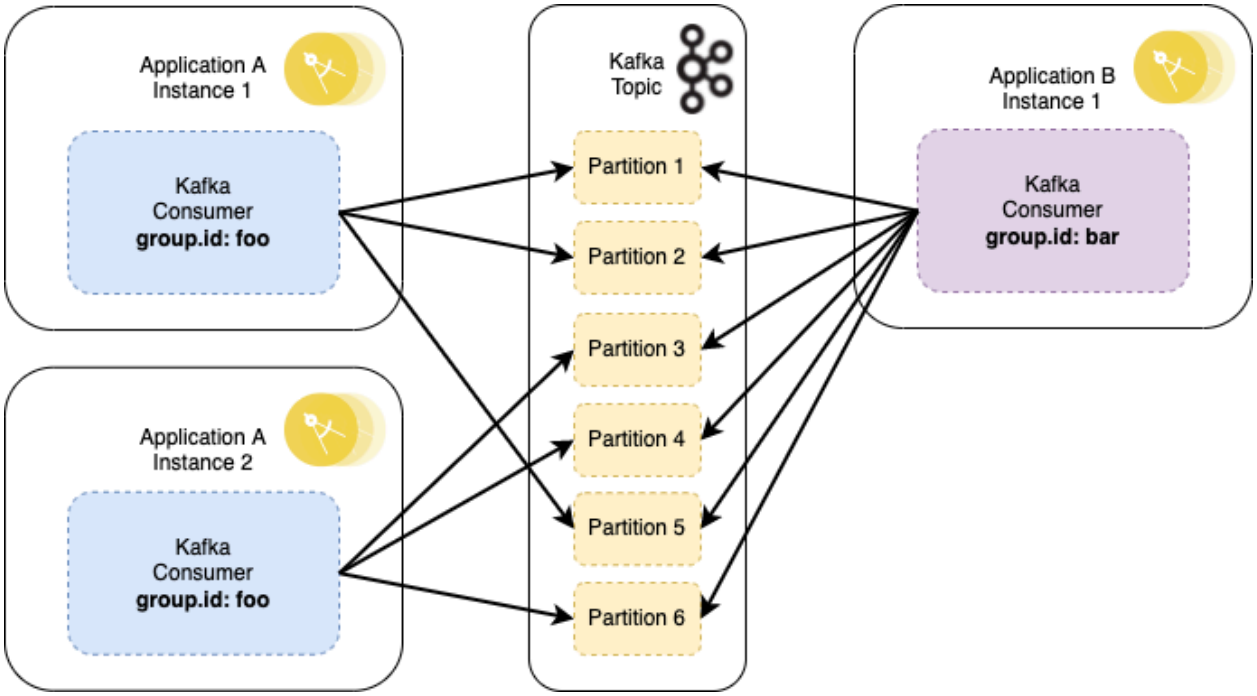


Figure 3: Two consumer groups 'foo' and 'bar'

Rebalance Triggers

There are several causes for a consumer group rebalance to take place. A new consumer joins a consumer group, an existing consumer leaves a consumer group, or the broker thinks a consumer may have failed. As well as these, any other need for resources to be reassigned will trigger a rebalance. An example is the creation of a topic where a consumer is configured with a pattern subscription that matches this topic name.

When a new consumer joins a consumer group it sends a JoinGroup request to the Group Coordinator on the broker. The topic partitions are then reassigned across all one or more consumers in the group. Likewise when a consumer leaves a group it notifies the Group Coordinator via a LeaveGroup request which again reassigns the topic partitions across the remaining consumers, if there are any.

When the Group Coordinator does not hear from a consumer within the expected timeframe, be it a heartbeat or the next poll() call, then it evicts the consumer from the group believing it may have failed. Once again the topic partitions are reassigned across any other consumers remaining in the group.

If a service has multiple consumers that listen to mutually exclusive topics but that share the same group.id then any rebalance triggered by any one consumer would still affect the other consumers in the group. In the following scenario Consumer A is consuming from

topic abc, whilst Consumer B consumes from topic def. They are in the same consumer group foo. If Consumer A takes too long to process a batch and times out then it is removed from the consumer group triggering a rebalance. All partition assignments in the group are revoked and reassigned, including those for Consumer B.

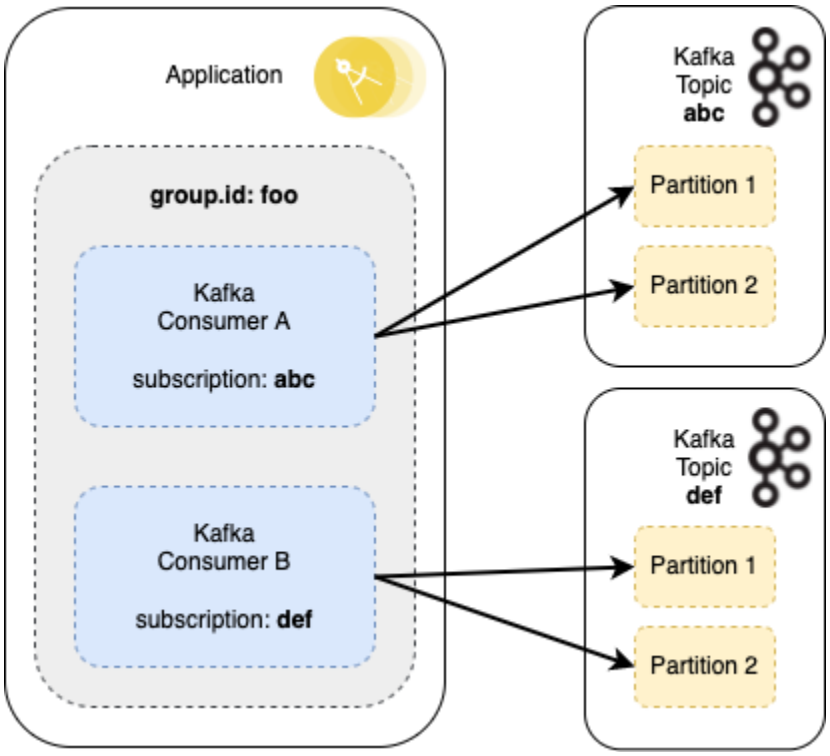


Figure 4: Consumer group spanning topics

When Consumer A eventually completes its poll and rejoins the consumer group, a further rebalance is triggered, and again all processing stops as partitions are revoked and reassigned. It can therefore be prudent to define separate consumer groups for consumers listening to different topics. e.g. *[service]-[topic]-consumer-group*.

Rebalance Configuration

Overview

For the Apache Java Kafka client the following are the key configurations on the consumer that impact how long rebalances can take to complete, and when a consumer may be considered to have failed by the broker causing a rebalance to be triggered.

Configuration	Description	Default
session.timeout.ms	The timeout to detect a consumer has failed. A heartbeat must be received by the Group Coordinator within this period.	45 seconds (from Kafka 3.0.0)
heartbeat.interval.ms	The interval between heartbeats sent by the consumer to the Group Coordinator. Used to ensure a consumer's session stays alive.	3 seconds
max.poll.interval.ms	The maximum time between poll invocations before a consumer is considered failed.	5 minutes

The following sections examine the impact of these configuration parameters.

Heartbeat and Session Timeout

The consumer sends periodic heartbeats to the Group Coordinator (which lives on the broker). This allows the Group Coordinator to monitor the health of the consumers in the group. A heartbeat must be received within the session.timeout.ms, and the heartbeats are sent based on the heartbeat.interval.ms. When the heartbeat is received by the Group Coordinator the session.timeout.ms resets, it responds to the consumer, and the next consumer heartbeat must be received within this reset timeout.

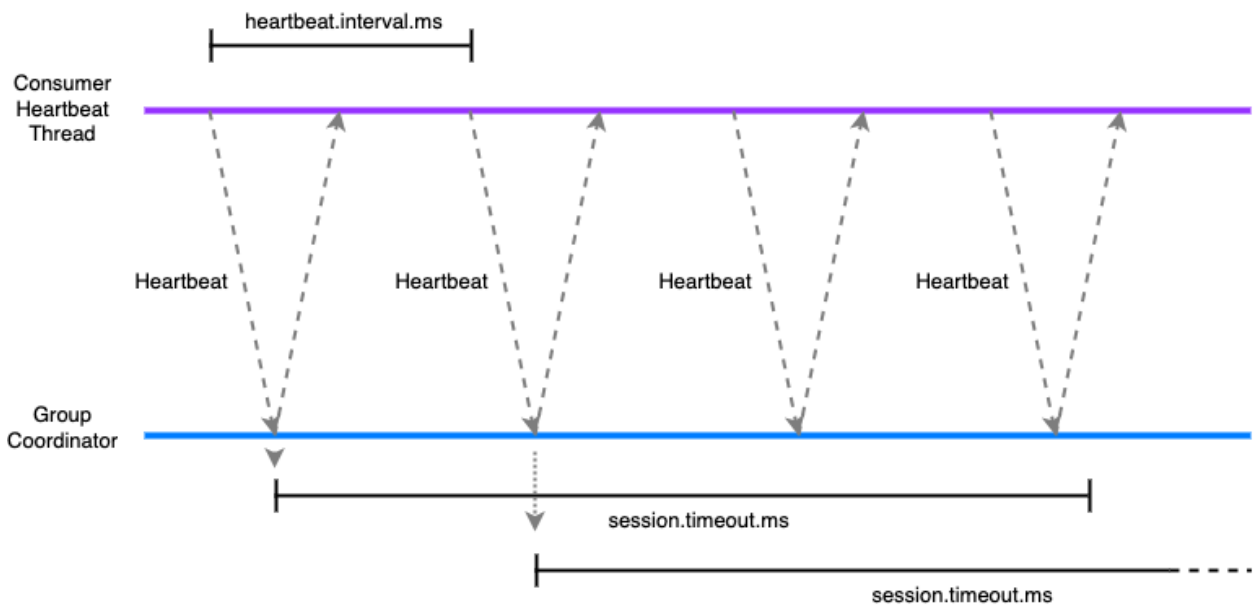


Figure 5: Consumer heart beating

It is recommended to configure the heartbeat.interval.ms to be no more than a third of session.timeout.ms. This ensures that if a heartbeat or two are lost due for example to a transient network issue, that the consumer is not considered to have failed. In this diagram two heartbeats are lost, but the third arrives before the session has timed out, so the Group Coordinator knows the consumer is still healthy.

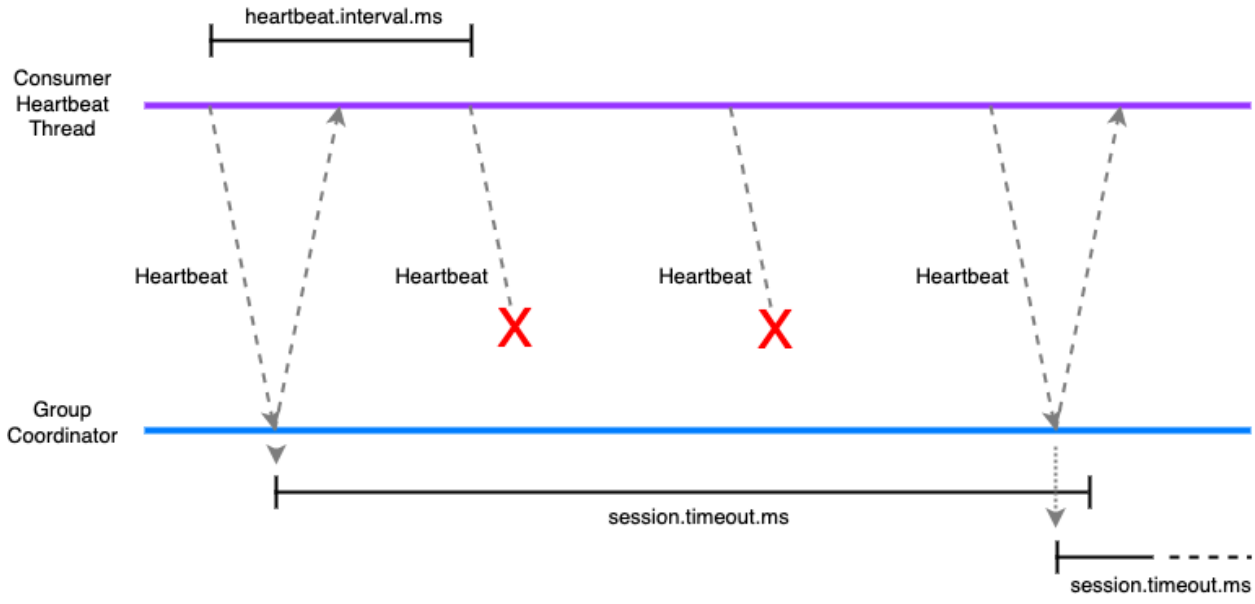


Figure 6: Failed heart beats

If the consumer does fail and stops heart beating then it is evicted from the consumer group once the session timeout expires, resulting in a consumer group rebalance.

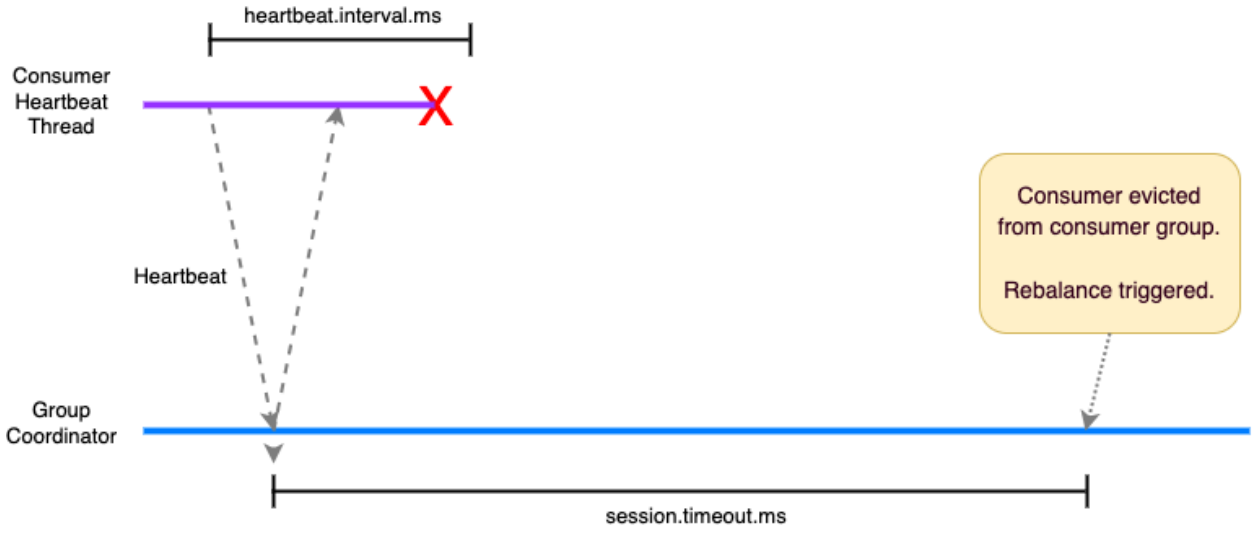


Figure 7: Consumer fails

Poll Interval

Heartbeats are performed on a separate thread to that of the main processing. The consumer polls its topic partitions on the main processing thread, and each call to poll() must happen within the configured max.poll.interval.ms. The following diagram adds the consumer processing thread, showing the responsibility of this thread alongside that of the heartbeat thread.

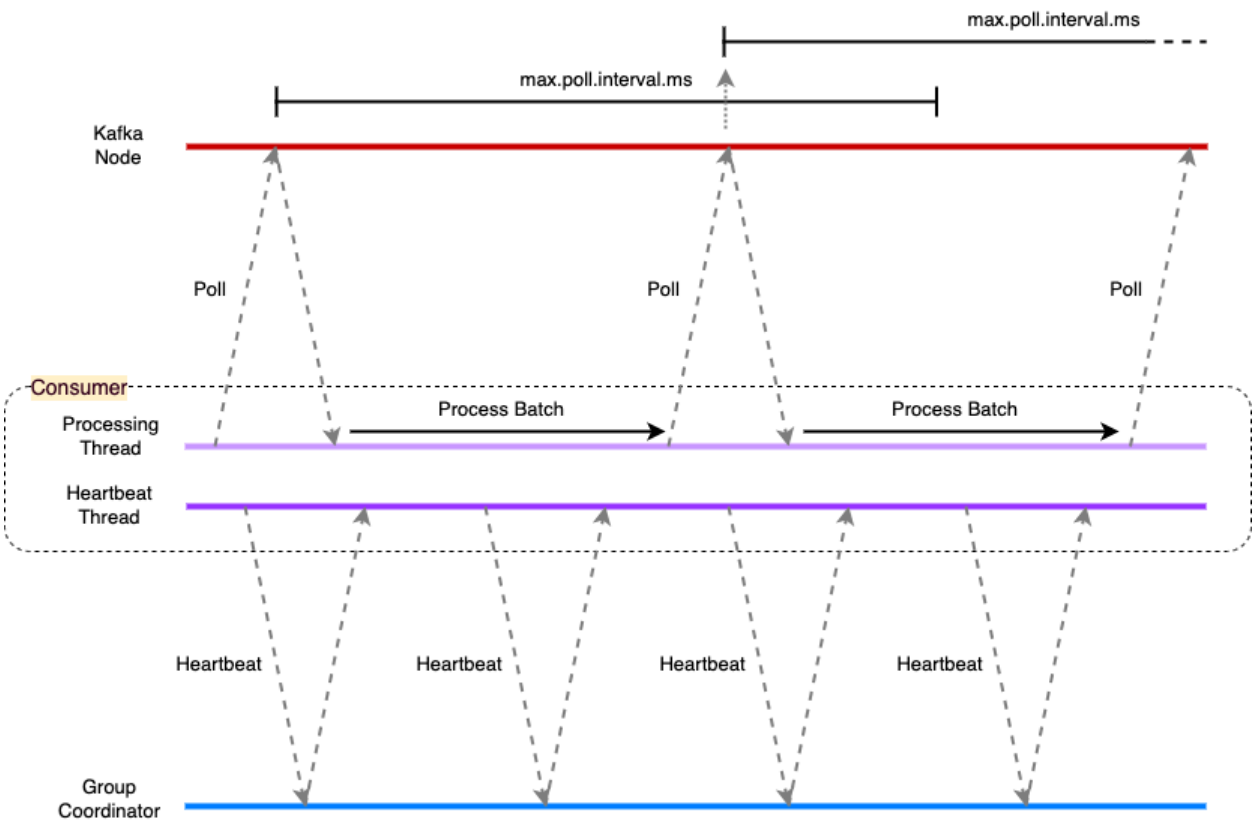


Figure 8: Consumer heart beating and polling

The first call to poll(), and any call to poll() that includes changes such as to partition assignments, results in the heartbeat thread being started. Each subsequent poll() call restarts the poll time, such that it has this full max.poll.interval.ms within which to complete.

The heartbeat thread checks the status of the consumer processing, and if the max.poll.interval.ms has been exceeded between polls then rather than a heartbeat it sends a LeaveGroup request. The Group Coordinator removes the consumer from the consumer group triggering a rebalance.

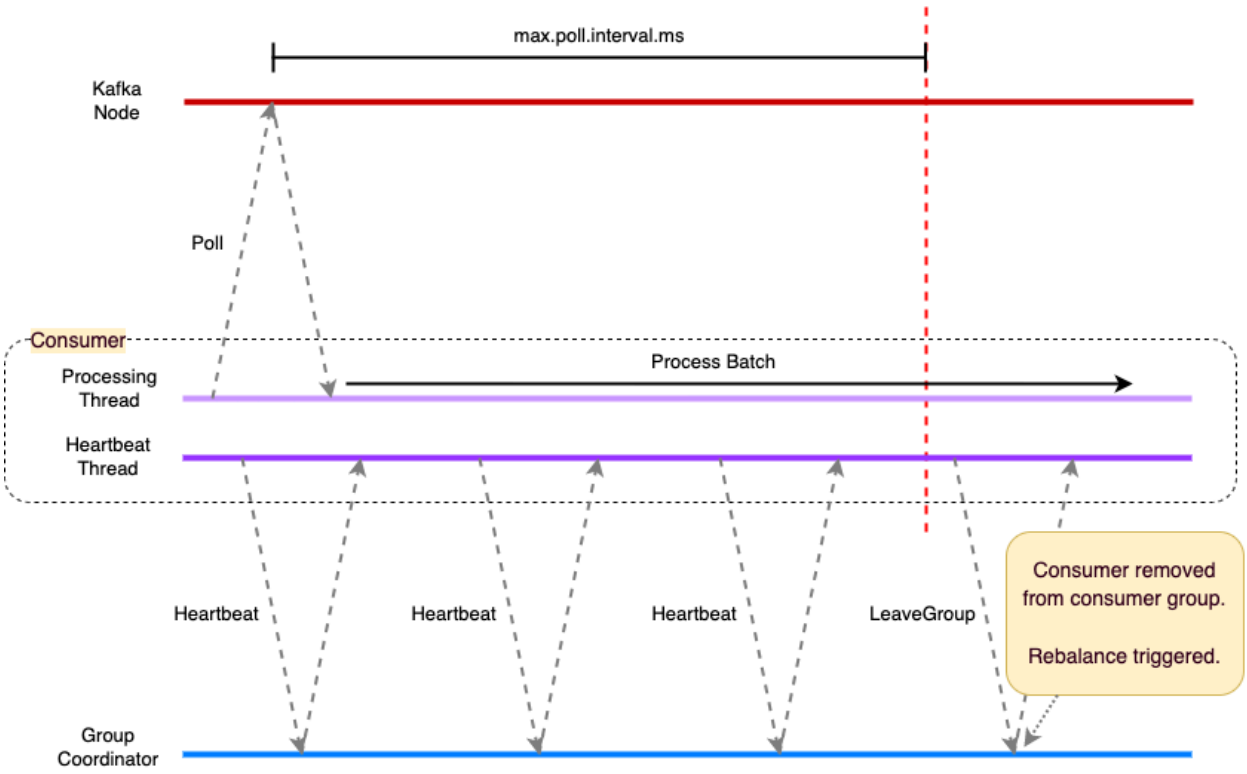


Figure 9: Consumer exceeds poll interval

When a rebalance is triggered the existing consumers will receive a response to their next heartbeat of 'Rebalance'. Each consumer has until the max.poll.interval.ms timeout to rejoin the group by calling poll(), as this triggers a JoinGroup request to the Group Coordinator. Note that for Kafka Connect a separate timeout is provided for this, rebalance.timeout.ms.

Configuring the max.poll.interval.ms therefore requires careful consideration. Set it too low and the risk is that the batch of messages consumed in a single poll are not processed in time leading to rebalancing and duplicate message delivery. Set the interval too high and it means that when a consumer does fail it takes longer before the broker is aware and the consumer's partitions are reassigned. During this processing the messages on the topic partitions assigned to the failed consumer are stuck.

Consumer Health

There are therefore two time outs to consider that have a bearing on when a consumer is considered healthy or to have failed and be evicted from a consumer group. If the main processing thread dies, leaving the heartbeat thread still running, the failure is detected by the max.poll.interval.ms being exceeded. If the whole application dies then this will be detected by no heartbeat being received within the session.timeout.ms.

The max.poll.interval.ms is essentially the main health check for the consumer processing. However by also utilising a heartbeat check on a separate thread it means that hard failures where the whole application has failed are detected more quickly.

Next...

In the second part of this article the rebalance strategies that can be applied and options to mitigate the effects of a rebalance are explored.